

터미널에서 만나는 클로드 코드

Claude Cowork*Skill

12장에서는 코워크^{Cowork}와 그 주변 도구를 둘러봤습니다. 이번 부록에서는 시야를 한 단계 더 안쪽으로 옮겨, 코워크와 같은 클로드 엔진을 터미널에서 사용하는 Claude Code를 살펴봅니다. 1장에서 “이 책은 터미널을 한 번도 열지 않는다”고 약속했고, 그 약속은 본문까지 지켰습니다. 이 부록은 터미널 사용을 강요하는 실습장이 아니라, 코워크 뒤에서 어떤 자동화가 가능한지 이해하기 위한 안내서입니다.

기본 명령어부터 권한 모드, 사고 깊이 조절, Sub Agent와 Agent Team, Hooks, 메모리, 병렬 작업, 플러그인까지 차례로 살펴보겠습니다. 모든 명령을 외울 필요는 없습니다. “이런 도구가 있다”는 감각만 익혀 두고, 실제로 필요할 때 다시 펼쳐 보면 충분합니다.

잠깐!

Check

[작성 기준] 이 부록은 Claude Code v2.1.220(2026년 7월 25일 공개)을 기준으로 2026년 7월 27일에 다시 작성했습니다. Claude Code는 업데이트 속도가 빠르므로 화면과 명령어가 달라졌다면 먼저 /help, /doctor, 공식 변경 이력을 확인하세요.

이 부록에서 다루는 내용

- 터미널에서 Claude Code를 시작하고 대화를 관리하는 기본 명령어
- Manual, Accepted edits, Plan, Auto 권한 모드의 차이
- /effort와 ultrathink로 사고 깊이를 조절하는 방법
- Sub Agent와 Agent Team의 구조, 활성화 방법, 사용 시 주의점
- Hooks로 작업 흐름에 자동 검사와 알림을 연결하는 방법
- CLAUDE.md, Auto-memory, 백그라운드 작업과 여러 세션
- bkit, Playwright MCP, /usage 같은 확장·관리 도구

1 기본 명령어 정리 터미널의 첫걸음

터미널에서 Claude Code를 사용할 때 가장 자주 만나는 명령어부터 살펴보겠습니다. 명령어의 모양보다 ‘언제 쓰는가’를 중심으로 보면 훨씬 쉽게 익힐 수 있습니다.

1.1 시작과 종료

터미널을 열고 작업할 프로젝트 폴더로 이동한 뒤 claude를 입력하면 대화형 세션이 시작됩니다. 자연어로 요청하는 방식은 코워크와 크게 다르지 않습니다.

명령어	하는 일	비유
claude	대화형 Claude Code 세션을 시작한다.	코워크 창을 새로 여는 것
claude “질문”	해당 질문을 첫 요청으로 넣은 대화형 세션을 시작한다. 답변 뒤에도 대화가 계속된다.	질문을 미리 적어 둔 코워크 창을 여는 것
claude -p “질문”	질문에 답한 뒤 종료한다. 스크립트나 자동화에서 유용하다.	한 번만 묻고 결과를 받는 호출
claude -c	현재 폴더에서 가장 최근 대화를 이어 간다.	방금 닫은 작업을 바로 다시 여는 것
claude --resume	목록에서 특정 세션을 골라 이어 간다. 이름이나 ID를 지정할 수도 있다.	예전 작업 목록에서 하나를 선택하는 것
/exit 또는 Ctrl+D	현재 세션을 종료한다.	창을 닫는 것
Ctrl+C 또는 Esc	진행 중인 응답이나 도구 실행을 중단한다.	작업을 잠시 멈추고 방향을 바꾸는 것

Tip

Ctrl+C는 “종료 전용 키”가 아닙니다. 작업 중에는 현재 실행을 멈추고, 아무 작업도 없을 때는 입력창을 비웁니다. 확실하게 종료하려면 /exit 또는 Ctrl+D를 사용하는 편이 좋습니다.

1.2 자주 쓰는 슬래시 명령어

세션 안에서 /를 입력하면 사용할 수 있는 명령어와 스킴이 나타납니다. 글자를 이어 입력하면 목록이 좁혀집니다. 설치한 플러그인이나 연결된 MCP 서버에 따라 보이는 항목은 달라질 수 있습니다.

명령어	하는 일	언제 쓰나
/init	프로젝트를 분석해 시작용 CLAUDE.md를 만든다.	새 프로젝트에서 처음 시작할 때
/memory	CLAUDE.md와 Auto-memory를 확인, 편집하고 자동 메모리를 켜거나 끈다.	프로젝트 규칙과 기억을 정리할 때
/permissions	도구의 allow, ask, deny 규칙과 작업 디렉터리를 관리한다.	승인 범위를 조절하거나 보안을 강화할 때
/config	테마, 모델, 출력 방식 등 설정 화면을 연다.	환경을 한눈에 조정할 때
/mcp	MCP 서버의 연결 상태와 인증을 관리한다.	외부 서비스 연결을 점검할 때
/agents	현재는 대화형 관리 화면이 아니라, 자연어 요청 또는 agents 폴더 편집 방법을 안내한다.	Sub Agent 생성 방법을 확인 할 때
/doctor	설치, 설정, 플러그인, Hooks, 컨텍스트 비용을 진단하고 수정안을 제시한다.	문제가 생겼거나 환경을 정리할 때
/help	현재 사용할 수 있는 도움말과 명령어를 보여 준다.	명령어가 기억나지 않을 때

잠깐! Check

[중요] /agents의 예전 Agents-Running-Library 대화형 화면은 v2.1.198부터 제거되었습니다. 현재는 “보안 검토용 Sub Agent를 만들어줘”처럼 자연어로 요청하거나 .claude/agents와 ~/.claude/agents의 마크다운 파일을 편집합니다.

1.3 대화와 컨텍스트 관리

대화가 길어지거나 작업이 여러 갈래로 나뉘면, 단순히 계속 입력하는 것보다 대화 상태를 관리하는 편이 안정적입니다.

명령어	하는 일	언제 쓰나
/clear	현재 대화를 빈 컨텍스트로 새로 시작한다.	주제가 완전히 바뀌었을 때
/compact	지금까지의 대화를 요약해 컨텍스트 공간을 확보한다.	대화는 이어가되 길이를 줄이고 싶을 때
/context	무엇이 컨텍스트를 차지하는지 시각적으로 보여 준다.	메모리·도구·대화 중 무엇이 큰지 확인할 때
/model	모델을 바꾸고 지원 모델에서는 effort도 조절한다.	속도·비용·추론 능력의 균형을 바꿀 때
/effort	모델이 문제에 들이는 추론 강도를 조절한다.	복잡한 분석이나 빠른 초안 사이를 전환할 때
/usage	세션 비용, 플랜 사용량, 활동 통계를 보여 준다. /cost와 /stats는 별칭이다.	현재 사용량을 확인할 때
/tasks	현재 세션의 백그라운드 Bash와 Sub Agent 작업을 확인하고 관리한다.	뒤에서 돌아가는 작업을 점검할 때

1.4 입력 접두어와 핵심 단축키

입력창 첫 글자와 키보드 단축키를 알면 터미널 작업이 훨씬 빨라집니다. 아래 항목만 기억해도 대부분의 상황을 처리할 수 있습니다.

입력 접두어	기능	설명
/	명령어·스킬	내장 명령어, 사용자 스킬, 플러그인 명령을 검색해 실행한다.
!	Shell 모드	명령어를 직접 실행하고 출력 내용을 대화에 포함한다. 예: !git status
@	파일 경로 참조	파일 경로 자동완성을 열어 특정 파일을 요청에 포함한다.
?	단축키 도움말	입력창이 비어 있을 때 도움말 패널을 켜고 끈다.

단축키	하는 일
Esc	현재 응답이나 도구 실행을 중단하거나 열린 대화상자를 닫는다.
Esc 두 번	입력이 비어 있으면 rewind 메뉴를 열고, 입력 중이면 초안을 비우되 기록에 보관한다.
Shift+Tab	Manual → Accept edits → Plan 순으로 활성화된 선택 모드를 순환한다.
Option+T(macOS) / Alt+T(Windows-Linux)	확장 사고(Extended Thinking)를 켜거나 끈다.
Ctrl+O	도구 사용과 실행 내역을 자세히 보는 transcript viewer를 연다.
Ctrl+B	실행 중인 Bash 명령어나 에이전트 작업을 백그라운드로 보낸다.
Ctrl+V·Cmd+V·Alt+V	터미널 환경에 맞는 키로 클립보드 이미지를 붙여넣는다.
Ctrl+L	화면을 다시 그린다. 일부 전체 화면 환경에서는 빠르게 두 번 누르면 /clear가 실행될 수 있다.

Tip

예전 자료에서 ‘&로 백그라운드 실행’, ‘#으로 CLAUDE.md에 기억 저장’, ‘Tab으로 사고 모드 전환’을 설명하는 경우가 있습니다. 현재 기준으로 백그라운드는 Ctrl+B, 메모리는 /memory 또는 자연어 요청, 확장 사고 토글은 Option+T 또는 Alt+T를 사용합니다.

1.5 /doctor — 문제가 생겼을 때 가장 먼저

/doctor는 설치 상태를 보여 주는 단순 화면이 아니라, 현재 Claude Code 환경을 점검하고 고칠 수 있는 진단 스킴입니다. 중복 설치, PATH 문제, 읽을 수 없는 설정 파일, 사용하지 않는 스킴·MCP·플러그인의 컨텍스트 비용, 느린 Hook, 업데이트 여부 등을 확인합니다.

진단 결과를 먼저 보여 주고, 실제 변경이 필요한 경우에는 확인을 받은 뒤 수정합니다. 세션을 열지 않고 설치 상태만 읽기 전용으로 보고 싶다면 터미널에서 `claude doctor`를 실행할 수 있습니다.

```
claude doctor
```

Tip

/doctor 화면은 버전마다 자주 바뀝니다. 화면에서 항목의 위치나 색을 외우기보다, “무엇이 문제인지 → 왜 문제인지 → 어떤 수정이 제안되는지” 세 가지를 읽는 습관이 더 오래갑니다.

1.6 CLAUDE.md — 프로젝트의 사내 규정집

/init을 실행하면 생성되는 CLAUDE.md는 프로젝트를 위한 지속 지침입니다. 코워크의 프로젝트 지침과 비슷하지만, Claude Code에서는 코드베이스와 함께 관리되는 핵심 문서입니다.

책 본문에서 SKILL.md를 ‘업무 매뉴얼’에 비유했습니다. CLAUDE.md는 ‘사내 규정집’에 가깝습니다. SKILL.md가 특정 작업의 절차를 설명한다면, CLAUDE.md는 프로젝트 전체에서 지켜야 할 언어, 명명 규칙, 테스트 방식, 금지 사항을 적습니다.

프로젝트 규칙

- 모든 설명과 보고서는 한국어로 작성한다.
- 패키지 관리는 npm이 아니라 pnpm을 사용한다.
- 기능을 수정하면 관련 테스트도 함께 갱신한다.
- 데이터베이스 스키마를 바꾸기 전에는 반드시 계획을 먼저 제시한다.
- 작업이 끝나면 변경 파일, 테스트 결과, 남은 위험을 요약한다.

Tip

CLAUDE.md는 ‘지침’이지 강제 보안 장치가 아닙니다. 어떤 명령을 무조건 막아야 한다면 deny 권한 규칙이나 PreToolUse Hook을 사용해야 합니다.

2 Claude Code의 권한 모드

얼마나 자율적으로 움직일까

권한 모드는 Claude가 파일을 고치거나 명령을 실행하기 전에 얼마나 자주 승인을 요청할지를 정합니다. 자동화의 기어보다는 ‘감독 강도’에 가깝습니다. 민감한 작업에서는 감독을 높이고, 방향이 확실한 반복 작업에서는 중단을 줄일 수 있습니다.

화면 이름	설정값	문지 않고 할 수 있는 범위	어울리는 상황
Manual	default	읽기 중심. 수정과 실행은 필요한 시점에 승인 요청	처음 시작할 때, 민감한 저장소
Accept edits	acceptEdits	읽기, 작업 범위 안의 파일 수정, 일부 일반 파일 명령	변경 내용을 git diff로 사후 검토할 때
Plan	plan	코드 탐색과 계획 작성. 소스 수정은 계획 승인 전 차단	큰 변경 전 영향 범위를 파악할 때
Auto	auto	별도 안전 분류기가 행동을 검사하며 일상적 승인 질문을 크게 줄임	방향을 신뢰하는 긴 작업

CLI에서는 Shift+Tab으로 Manual, Accept edits, Plan을 순환하며, 계정과 모델이 지원하면 Auto도 순환 목록에 나타납니다. 시작할 때 특정 모드로 열려면 --permission-mode를 사용합니다.

```
claude --permission-mode plan
claude --permission-mode acceptEdits
claude --permission-mode auto
```

2.1 Manual — 가장 안전한 기본값

Manual은 Claude가 읽기 외의 행동을 시도할 때 사용자가 흐름을 확인하도록 만드는 모드입니다. 처음 보는 저장소, 운영 환경과 연결된 프로젝트, 중요한 문서를 다룰 때 가장 적합합니다. 승인 질문이 많아 속도는 느리지만, ‘무엇을 하려는지’를 배우기에도 좋습니다.

2.2 Accept edits — 파일 수정은 말기고 결과를 검토

Accept edits는 작업 디렉터리 안의 파일 생성과 수정, 일부 일반적인 파일 명령을 자동 승인합니다. 모든 Bash 명령을 무조건 허용하는 모드는 아닙니다. 범위를 벗어난 경로, 보호된 경로, 그 밖의 명령은 여전히 승인을 요청할 수 있습니다.

짧은 기능 구현이나 리팩터링처럼 방향이 분명하고, 수정 후 git diff와 테스트 결과를 검토할 수 있을 때 유용합니다.

2.3 Plan — 먼저 조사하고, 나중에 수정

Plan은 Claude가 코드베이스를 읽고 필요한 명령으로 탐색한 뒤 계획을 제시하도록 합니다. 계획을 승인하기 전에는 소스 파일을 수정하지 않습니다. 처음 보는 프로젝트나 구조를 크게 바꾸는 작업에서 “바로 고치지 말고 현재 상태와 영향 범위부터 확인해줘”라고 요청할 때 잘 맞습니다.

/plan 인증 구조를 OAuth 2.1 기반으로 바꿀 때 영향 범위와 단계별 계획을 작성해줘

계획이 제시되면 그대로 승인하거나, 수정을 요청하거나, 계속 계획 단계에 머무를 수 있습니다. 계획을 승인한 뒤에는 선택한 권한 모드로 전환되어 실제 구현이 시작됩니다.

2.4 Auto — 질문을 줄이되 검토를 없애지는 않기

Auto는 일상적인 권한 질문을 줄이고 긴 작업을 이어 가도록 돕습니다. 실행 전에는 별도의 안전 분류기가 요청 범위를 벗어나는 행동, 알 수 없는 인프라, 외부 콘텐츠에 유도된 위험 행동 등을 검사합니다. 다만 ‘자동’이 ‘완전 안전’을 뜻하지는 않습니다.

잠깐!
Check

[중요] Auto는 계정 정책, 선택 모델, 제공 환경에 따라 보이지 않을 수 있습니다. Team이나 Enterprise에서는 조직 관리자가 허용해야 할 수 있으며, 지원되지 않는 모델에서는 사용할 수 없습니다. 민감한 운영·배포·권한 변경은 Auto에 맡겼더라도 반드시 사람이 검토해야 합니다.

2.5 권장 워크플로

흐름	언제 쓰나	진행 방법
Manual → Plan → Manual	처음 보는 중요한 프로젝트	질문을 확인하며 탐색 → 계획 검토 → 수정도 하나씩 승인
Plan → Accept edits	일반적인 기능 추가, 리팩터링	계획을 먼저 검토 → 파일 수정은 자동 → diff와 테스트를 최종 확인
Plan → Auto	경계가 분명한 긴 작업	완료 기준과 금지 범위를 명확히 적고 계획 승인 → 중간 결과를 주기적으로 점검

Tip

처음에는 Manual과 Plan을 중심으로 사용하세요. Claude가 어떤 명령을 선택하고 어떤 방식으로 코드를 수정하는지 익숙해진 뒤 Accept edits나 Auto로 이동하는 편이 안전합니다.

3 사고 깊이 조절 /effort와 ultrathink

권한 모드가 ‘무엇을 실행해도 되는가’를 정한다면, effort는 ‘얼마나 깊게 생각할 것인가’를 조절합니다. 서로 다른 기능이므로 혼동하지 않아야 합니다.

3.1 /effort — 세션의 기본 사고 강도

/effort는 현재 모델이 지원하는 범위 안에서 추론 강도를 바꿉니다. 보통 low, medium, high, xhigh가 쓰이며, 모델과 환경에 따라 max나 ultracode가 추가될 수 있습니다. /effort만 입력하면 선택 화면이 열리고, /effort auto는 모델 기본값으로 돌아갑니다.

```
/effort low
/effort high
/effort auto
```

수준	어울리는 작업	특징
low	형식 변환, 간단한 문구 수정, 명확한 반복 작업	빠르고 가볍지만 복잡한 판단에는 부족할 수 있음
medium·auto	일반적인 구현, 문서 작성, 보통 난도의 분석	속도와 품질의 균형
high·xhigh	원인 분석, 시스템 설계, 여러 조건이 얹힌 의사결정	더 많은 시간과 토큰을 사용할 수 있음
max·ultracode	지원 모델에서 매우 복잡하거나 대규모인 작업	세션 전용이거나 이용 가능 범위가 제한될 수 있음

3.2 ultrathink — 한 번의 요청만 더 깊게

세션 전체 effort를 바꾸지 않고 특정 요청만 깊게 검토하고 싶다면 프롬프트 어디에든 ultrathink를 포함할 수 있습니다. Claude Code는 이 단어를 특별한 지시로 인식해 해당 턴에 더 깊은 검토를 요청합니다.

```
ultrathink. 이 마이그레이션 계획에서 롤백이 실패할 수 있는 경로를 반례 중심으로 검토해줘.
```



[중요] “think”, “think hard”, “think more”는 현재 Claude Code에서 특별한 단계 키워드로 인식되지 않습니다. 일반 문장으로 전달될 뿐입니다. 특별 키워드는 ultrathink이며, 지속적인 강도 조절은 /effort를 사용합니다.

3.3 확장 사고 토클과의 차이

Option+T(macOS) 또는 Alt+T(Windows·Linux)는 확장 사고를 켜거나 끄는 단축키입니다. /effort는 켜진 사고가 어느 정도의 강도로 작동할지를 조절하고, ultrathink는 한 번의 요청에 깊은 검토 신호를 추가합니다.

기능	범위	사용 방법
확장 사고 토클	세션에서 사고 기능을 켜고 끄	Option+T 또는 Alt+T
/effort	세션의 기본 추론 강도	/effort high 등
ultrathink	현재 한 번의 요청	프롬프트에 ultrathink 포함



간단한 작업에 무조건 높은 effort나 ultrathink를 쓰면 응답 시간과 토큰 사용만 늘 수 있습니다. 먼저 보통 강도로 시도하고, 실패 원인이 추론 부족일 때 높이는 편이 효율적입니다.

Claude Code는 한 세션 안에서 보조 작업자를 호출하는 Sub Agent와 여러 독립 세션이 팀처럼 협력하는 Agent Team을 제공합니다. 둘 다 ‘여러 Claude가 일한다’는 점은 비슷하지만, 통신 구조와 적합한 작업이 다릅니다.

4.1 Sub Agent — 전문 보조 작업자에게 위임하기

Sub Agent는 메인 대화가 특정 역할을 가진 보조 에이전트에게 일부 작업을 맡기고 결과를 돌려받는 구조입니다. 메인 대화의 컨텍스트를 지저분하게 만들지 않고 코드 조사, 테스트 실행, 리뷰, 로그 분석 같은 집중 작업을 분리하기 좋습니다.

현재는 예전 /agents 대화형 생성 화면을 사용하지 않습니다. 가장 쉬운 방법은 자연어로 필요한 에이전트를 만들어 달라고 요청하는 것입니다.

보안 취약점만 검토하고 파일은 수정하지 않는 Sub Agent를 프로젝트용으로 만들어줘.

직접 만들고 싶다면 마크다운 파일을 다음 위치에 둡니다.

저장 위치	범위	특징
.claude/agents/	현재 프로젝트	Git에 포함해 팀원과 공유하기 좋음
~/.claude/agents/	내 모든 프로젝트	개인 공통 에이전트로 사용

예를 들어 읽기 전용 보안 리뷰어는 다음처럼 정의할 수 있습니다. 실제 frontmatter 항목과 도구 이름은 버전에 따라 확장될 수 있으므로, 처음에는 Claude에게 파일 생성을 맡기고 결과를 검토하는 방식이 편합니다.

```
---
name: security-reviewer
description: 인증 · 권한 변경이 있을 때 보안 위험을 검토한다.
tools: Read, Grep, Glob
model: sonnet
---
```

변경된 인증 코드를 읽고 취약점, 권한 상승 가능성, 민감정보 노출을 검토한다.
파일을 수정하지 말고 근거와 심각도를 보고한다.

Tip

Sub Agent에는 필요한 도구만 허용하세요. 코드 리뷰어라면 Read, Grep, Glob처럼 읽기 도구로 제한하고, 구현 작업에만 Edit, Write를 허용하면 실수 범위를 줄일 수 있습니다.

4.2 Sub Agent와 Agent Team의 차이

구분	Sub Agent	Agent Team
구조	한 메인 세션 안에서 보조 작업자가 실행됨	리드 1명과 여러 독립 Claude Code 세션
통신	작업 결과를 메인 에이전트에 반환	팀원끼리 직접 메시지를 주고받을 수 있음
사용자 상호작용	주로 메인 대화와 상호작용	리드뿐 아니라 개별 팀원 세션에도 직접 지시 가능
공유 자원	Agent Team의 공유 작업 목록, 메일박스 없음	공유 작업 목록과 팀원 간 메시지
적합한 작업	짧고 집중된 조사·검증·보조 작업	여러 관점의 연구, 독립 모듈, 경쟁 가설 디버깅
비용·조율	상대적으로 가벼움	각 팀원이 별도 컨텍스트를 써 비용과 조율 부담이 큼

한 명의 보조 전문가가 조사한 뒤 메인에게 보고하는 것으로 충분하다면 Sub Agent가 알맞습니다. 팀원끼리 발견을 공유하고 서로 반박하거나 프론트엔드와 백엔드, 테스트처럼 독립 영역을 동시에 진행해야 한다면 Agent Team이 더 적합합니다.

4.3 Agent Team 활성화 — 실험 기능을 먼저 켜기



[중요] Agent Team은 실험 기능이며 기본적으로 꺼져 있습니다. 다음과 같이 설정하지 않으면 팀 디렉터리가만 들어지지 않고, Claude가 팀원을 생성하거나 제안하지 않습니다.

사용자 설정 또는 프로젝트 설정의 settings.json에 다음 환경 변수를 추가합니다.

```
{
  "env": {
    "CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS": "1"
  }
}
```

Shell 환경 변수로 켤 수도 있습니다.

```
export CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1
```

설정 후에는 원하는 팀 역할을 자연어로 분명하게 요청합니다.

이 PR을 Agent Team으로 리뷰해줘.

- security: 인증과 권한 위험
- performance: 병목과 불필요한 I/O
- test: 누락된 테스트와 경계 조건

세 팀원이 발견을 서로 공유하고 반박한 뒤, 리드가 공통 결론과 이견을 정리해줘.

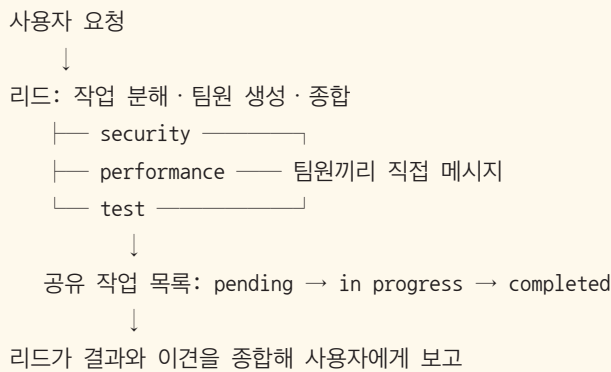
Tip

Claude가 팀 대신 Sub Agent를 생성할 때가 있습니다. 둘은 같은 에이전트 패널에 나타날 수 있으므로 패널만 보고 팀 구성 여부를 확정하기 어렵습니다. 이 경우 “Sub Agent가 아니라 Agent Team을 구성해줘”라고 명시적으로 다시 요청하세요.

4.4 Agent Team이 실제로 동작하는 구조

Agent Team이 구성되면 현재 메인 세션이 리드^{lead}가 됩니다. 리드는 작업을 쪼개고 팀원을 생성하고 업무를 배정하며 마지막 결과를 종합합니다. 각 팀원은 별도의 컨텍스트 윈도우를 가진 독립 Claude Code 세션입니다.

팀원은 CLAUDE.md와 MCP 서버, 스킬 같은 프로젝트 컨텍스트를 불러오지만 리드가 지금까지 나눈 전체 대화 내용은 그대로 물려받지 않습니다. 따라서 팀원을 만들 때 담당 범위, 파일 위치, 배경 정보, 완료 기준을 충분히 넣어야 합니다.



작업은 리드가 특정 팀원에게 배정할 수도 있고, 팀원이 완료 후 미할당 작업을 스스로 가져갈 수도 있습니다. 팀원 들은 이름으로 서로 직접 메시지를 보낼 수 있으며, 사용자는 에이전트 패널에서 개별 팀원을 선택해 추가 지시를 내릴 수 있습니다.

4.5 화면 표시 방식

방식	특징	환경
In-process	한 터미널 안의 에이전트 패널에서 팀원을 선택해 세션을 확인하고 메시지를 보냄	모든 터미널에서 동작하는 기본 방식
Split panes	팀원마다 별도 패널을 열어 출력을 동시에 확인	tmux 또는 iTerm2와 추가 설정 필요

처음에는 별도 설치가 필요 없는 In-process 방식으로 충분합니다. 한 화면에서 팀원 목록과 공유 작업을 확인하고, 필요한 팀원을 선택해 방향을 수정할 수 있습니다.

4.6 내부 조율 도구 — 사용자가 직접 호출할 필요는 없음

Agent Team은 내부적으로 팀원 생성, 메시지, 공유 작업 목록을 위한 도구를 사용합니다. 그러나 일반 사용자가

도구 이름을 함수처럼 직접 호출할 필요는 없습니다. “보안 팀원에게 테스트 팀원의 발견을 전달해줘”처럼 리드에게 자연어로 지시하면 됩니다.

내부 도구	역할	주의
Agent	팀원을 생성한다. 첫 팀원이 생성되면 메인 세션이 리드가 된다.	별도의 TeamCreate 단계가 없음
SendMessage	팀원 간 일반 메시지, 계획 승인, 종료 요청 등 조율 메시지를 전달한다.	사용자는 자연어로 지시하면 됨
TaskCreate, TaskUpdate	공유 작업을 만들고 pending, in progress, completed 상태를 갱신한다.	완료 상태가 늦게 반영될 수 있음
TaskList, TaskGet	공유 작업 목록과 특정 작업의 상세 상태를 조회한다.	누가 무엇을 맡았는지 확인하는 기반



[중요] v2.1.178부터 TeamCreate와 TeamDelete는 존재하지 않습니다. 팀원 생성 시 별도 팀 설정 단계가 필요 없으며, 세션이 끝나면 공유 팀 디렉터리도 자동으로 정리됩니다. 예전 자료의 Teammate, TeamCreate, TeamDelete를 그대로 사용하면 안 됩니다.

4.7 Agent Team을 쓰기 알맞은 작업과 피해야 할 작업

잘 맞는 작업	이유
여러 관점의 연구·리뷰	팀원들이 각 관점을 독립적으로 조사한 뒤 서로 발견을 검증할 수 있음
경쟁 가설 디버깅	각 팀원이 서로 다른 원인 가설을 시험해 한 방향에 고정되는 것을 줄임
프론트엔드, 백엔드, 테스트처럼 파일이 분리된 작업	담당 경계가 분명해 병렬 작업의 이점이 큼
독립 모듈을 동시에 만드는 신규 기능	팀원별 파일 소유권을 나누기 쉬움

피해야 할 작업	이유
순서대로 해야 하는 작업	앞 단계 결과를 기다리는 시간이 많아 병렬화 이점이 적음
여러 팀원이 같은 파일을 고치는 작업	덮어쓰기가 발생할 수 있음
의존성이 매우 많은 작은 작업	조율 비용이 실제 작업보다 커질 수 있음
한 사람이 금방 끝낼 수 있는 일	여러 컨텍스트의 토큰 비용만 늘어남

4.8 Agent Team 사용 시 주의할 점

주의할 점	설명
토큰 사용 증가	각 팀원이 별도 컨텍스트를 사용하므로 활성 팀원 수에 따라 비용이 대체로 선형으로 증가한다.
권장 규모	대부분의 작업은 팀원 3~5명부터 시작한다. 실제 병렬성이 있을 때만 늘린다.
파일 충돌	팀원마다 담당 파일을 분리한다. 같은 파일 편집은 덮어쓰기 위험이 있다.
세션 복원 제한	In-process 팀원은 /resume, /rewind로 복원되지 않는다. 재개 후 새 팀원을 생성해야 할 수 있다.
상태 지연	팀원이 끝났어도 작업 상태가 완료로 바뀌지 않을 수 있다. 실제 결과를 확인하고 리드에게 갱신을 요청한다.
종료 지연	팀원은 현재 요청이나 도구 호출을 마친 뒤 종료하므로 시간이 걸릴 수 있다.
한 세션에 한 팀	세션마다 팀 하나만 있으며 리드는 바꿀 수 없다.
중첩 팀 불가	팀원은 자신의 팀원을 생성할 수 없고, 팀 관리는 리드만 한다.

Tip

처음 Agent Team을 사용할 때는 코드 구현보다 PR 리뷰, 라이브러리 조사, 버그 원인 조사처럼 담당 경계가 명확하고 파일을 고치지 않는 작업부터 시작하는 것이 좋습니다.

4.9 10장의 오케스트레이션과 비교

10장에서 살펴본 오케스트레이션은 한 Claude가 이미지 생성기, 음성 생성기, MCP 서버 같은 여러 도구를 호출해 결과를 연결하는 구조였습니다. Agent Team은 여러 Claude Code 세션이 역할을 나누어 협력하는 구조입니다.

구분	도구 오케스트레이션	Agent Team
핵심	한 에이전트가 여러 외부 도구를 호출	여러 독립 Claude Code 세션이 협력
작업자	Claude + 외부 AI, API, MCP 도구	리드 + 여러 Claude Code 팀원
통신	도구 결과가 주로 메인 에이전트로 돌아옴	팀원끼리 발견과 지시를 직접 교환
강점	전문 외부 서비스의 기능을 조합	독립적인 판단과 병렬 탐색
주의	도구 연결과 순서 설계	토큰 비용, 파일 충돌, 조율 비용

5 Hooks

작업 흐름에 자동 규칙 걸기

Hooks는 Claude Code에서 특정 사건이 발생할 때 자동으로 명령, 프롬프트, 에이전트 또는 HTTP 요청을 실행하는 장치입니다. 이름 그대로 작업 흐름에 갈고리를 걸어 검사, 포매팅, 알림, 차단 규칙을 붙입니다.

5.1 예약 작업과 Hooks의 차이

구분	예약·반복 작업	Hooks
트리거	시간이나 반복 주기	Claude Code 안에서 발생한 이벤트
비유	정해진 시간에 울리는 알람	사람이 지나갈 때 켜지는 센서등
예시	5분마다 배포 상태 확인	Bash 실행 전 위험 명령 검사
목적	나중 또는 반복 실행	현재 작업 흐름을 검사·보강·기록

5.2 Pre와 Post만 있는 것이 아닙니다

Hooks를 ‘작업 전 Pre Hook, 작업 후 Post Hook’ 두 종류로만 설명하면 실제 기능을 지나치게 좁히게 됩니다. Claude Code에는 세션 시작, 사용자 입력, 도구 실행 전후, 작업 종료, 압축, 팀원 상태 등 여러 이벤트가 있습니다.

이벤트	언제 실행되나	대표 활용
SessionStart	세션이 시작될 때	환경 확인, 공통 정보 불러오기
UserPromptSubmit	사용자 요청이 제출된 직후	요청 기록, 규칙 보강

이벤트	언제 실행되나	대표 활용
PreToolUse	도구가 실행되기 직전	위험 명령·보호 파일 수정 차단
PermissionRequest	권한 승인이 필요할 때	정책에 따른 자동 판단 또는 기록
PostToolUse	도구가 성공한 직후	포매터, 린터, 후속 검사
PostToolUseFailure	도구가 실패한 직후	오류 로그 수집, 알림
Stop	Claude가 현재 작업을 끝내려 할 때	완료 기준 검증, 요약 저장
SessionEnd	세션이 끝날 때	감사 로그, 정리 작업
TeammateIdle, TaskCompleted	Agent Team의 팀원이나 작업 상태가 바뀔 때	품질 게이트, 완료 조건 검사

5.3 간단한 예시 — 파일 수정 후 포매터 실행

다음 예시에서는 Claude가 Edit 또는 Write 도구로 파일을 고친 뒤 Prettier를 실행합니다. `.claude/settings.json`에 넣는 프로젝트용 설정 예시입니다. Node.js, jq, Prettier가 준비된 환경을 전제로 합니다.

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "jq -r '.tool_input.file_path' | xargs npx prettier --write"
          }
        ]
      }
    ]
  }
}
```

/hooks를 실행하면 현재 설정된 Hook을 이벤트별로 확인할 수 있습니다. Hook이 실행되지 않는다면 이벤트 이름, matcher의 대소문자와 도구 이름, 스크립트 실행 권한을 먼저 점검합니다.

5.4 Hooks는 지침보다 강한 장치

CLAUDE.md에 “.env 파일을 수정하지 마”라고 적는 것은 행동을 안내하는 지침입니다. 반면 PreToolUse Hook에서 .env 수정을 차단하면 해당 도구 호출 자체를 금지할 수 있습니다. 반드시 지켜야 하는 보안 규칙이나 품질 게이트에는 Hook이 더 적합합니다.



[중요] Hook의 command는 사용자의 권한으로 실제 셸 명령을 실행할 수 있습니다. 인터넷에서 받은 Hook과 플러그인을 그대로 설치하지 말고, 어떤 명령과 경로를 사용하는지 확인하세요. 잘못 작성한 Hook은 정상 작업을 막거나 민감정보를 외부로 보낼 수 있습니다.

6 기억과 병렬 작업 CLAUDE.md, Auto-memory, 여러 세션

Claude Code의 각 대화는 새로운 컨텍스트에서 시작하지만, 프로젝트 지침과 자동 메모리를 통해 세션 사이에 지식을 이어 갈 수 있습니다. 동시에 여러 작업을 실행할 때는 기억의 범위와 파일 충돌을 함께 관리해야 합니다.

6.1 CLAUDE.md와 Auto-memory의 역할

구분	CLAUDE.md	Auto-memory
누가 작성하나	사용자가 작성하고 검토	Claude가 사용자의 수정과 반복 패턴을 바탕으로 기록
내용	규칙, 워크플로, 아키텍처, 금지 사항	빌드 명령, 디버깅 발견, 선호도와 반복 학습
용도	명확하고 지속적인 행동 지침	대화에서 자연스럽게 발견된 지식 보관
관리	/memory 또는 파일 편집	/memory에서 확인·편집·삭제·켜기/끄기

“항상 pnpm을 사용한다”, “운영 DB에 직접 접속하지 않는다”처럼 중요한 규칙은 CLAUDE.md에 직접 적습니다. “이 프로젝트의 테스트는 로컬 Redis가 필요하다”처럼 작업 중 발견한 사실은 Auto-memory가 기록하기 좋습니다.

두 메모리 모두 Claude에게 제공되는 컨텍스트이므로 절대적인 보안 정책은 아닙니다. 강제 차단이 필요하다면 권한 규칙이나 Hook을 사용합니다.

6.2 /memory로 기억 확인하기

/memory를 실행하면 사용자와 프로젝트 범위의 CLAUDE.md 위치를 확인하고, Auto-memory를 켜거나 끄고, 자동 메모리 파일을 열어 수정할 수 있습니다. Claude에게 “앞으로 이 프로젝트에서는 Jest가 아니라 Vitest를 사용한다는 점을 기억해”라고 요청하면 Auto-memory에 기록할 수 있습니다.

Tip

Auto-memory가 틀린 내용을 기억하면 이후 작업에도 반복될 수 있습니다. 프로젝트의 도구나 규칙이 바뀌었을 때는 /memory로 오래된 기록을 정리하세요.

6.3 한 세션 안에서 백그라운드 작업

오래 걸리는 테스트나 서버 실행을 기다리면서 다른 대화를 이어 가고 싶다면 Ctrl+B로 Bash 명령이나 에이전트 작업을 백그라운드로 보낼 수 있습니다. 진행 중인 작업은 /tasks에서 확인합니다.

백그라운드 작업은 같은 대화의 일부입니다. Claude는 작업 ID와 출력 파일을 통해 결과를 다시 읽을 수 있습니다. 단, 세션을 완전히 종료하면 일반 백그라운드 작업은 정리될 수 있으므로 장시간 유지가 필요한 경우 세션 자체를 백그라운드로 전환하는 방식을 검토합니다.

6.4 세션 자체를 백그라운드로 보내기

`/background` 또는 `/bg`는 현재 대화를 백그라운드 세션으로 분리해 터미널을 비웁니다. 실행 중인 백그라운드 세션은 `claude agents`에서 확인하고 다시 연결할 수 있습니다.

```
/background 테스트가 끝날 때까지 실패 원인을 분석하고 수정안을 정리해줘

# 새 터미널에서 백그라운드 세션 목록 확인
claude agents
```

6.5 여러 터미널 세션과 파일 충돌

터미널 탭을 여러 개 열어 서로 다른 Claude Code 세션을 동시에 사용할 수도 있습니다. 각 세션은 대화 컨텍스트가 독립적이지만 같은 프로젝트의 `CLAUDE.md`를 공유하므로 공통 규칙을 맞추기 좋습니다.

다만 여러 세션이 같은 파일을 동시에 수정하면 충돌이나 덮어쓰기가 생길 수 있습니다. 병렬 구현이 필요하면 담당 디렉터리나 파일을 나누고, 규모가 크다면 `Git worktree`를 사용해 각 세션에 별도 작업 트리를 주는 편이 안전합니다.

상황	권장 방식
자료 조사와 코드 구현을 동시에	서로 다른 세션 또는 Sub Agent
테스트를 오래 돌리며 다른 수정 진행	<code>Ctrl+B + /tasks</code>
한 대화를 터미널 없이 계속 실행	<code>/background + claude agents</code>
같은 저장소에서 여러 기능 병렬 구현	담당 파일 분리 + <code>Git worktree</code>
팀원끼리 직접 협의가 필요한 복잡한 병렬 작업	Agent Team

7 플러그인과 확장 기능

필요한 능력을 묶어 추가하기

Claude Code 플러그인은 스킴, Sub Agent, Hook, MCP 서버, LSP 서버, 모니터 등을 하나의 패키지로 묶어 기능을 확장합니다. `/plugin`을 실행하면 설치부터 목록, 활성화/비활성화를 관리할 수 있습니다.

명령어	하는 일
<code>/plugin</code>	플러그인 관리 화면을 연다.
<code>/plugin list</code>	설치된 마켓플레이스 플러그인을 확인한다.
<code>claude plugin install 이름@마켓</code>	플러그인을 설치한다.
<code>claude plugin details 이름</code>	포함된 스킴·에이전트·Hook·MCP와 예상 컨텍스트 비용을 확인한다.
<code>claude plugin disable 이름</code>	삭제하지 않고 플러그인을 끈다.
<code>claude plugin update 이름</code>	최신 버전으로 갱신한다.



[중요] 플러그인은 코드와 Hook, MCP 서버를 포함할 수 있습니다. 이름이 유명하다는 이유만으로 설치하지 말고 출처와 저장소, 라이선스, 실행 명령, 요청 권한을 확인하세요. 프로젝트 범위 플러그인은 함께 작업하는 사람에게도 영향을 줄 수 있습니다.

7.1 bkit — 제3자 AI 개발 워크플로

bkit은 Anthropic 공식 기능이 아니라 popup-studio-ai가 개발한 제3자 Claude Code 플러그인입니다. PDCA, Sprint, 전문 에이전트, 품질 게이트, 세션 간 메모리를 묶어 큰 개발 작업을 단계적으로 진행하도록 돕습니다.

현재 저장소가 소개하는 핵심 진입점은 /sprint, /pdca, /control 세 명령입니다. 기능 수와 내부 구성은 빠르게 바뀌므로, 세부 에이전트 수나 단계는 설치한 버전의 README와 /plugin details를 확인하는 것이 좋습니다.

명령	용도	쉽게 말하면
/sprint	여러 기능이 연결된 릴리스와 장기 작업을 나눠 관리	큰 프로젝트를 세션 크기에 맞게 쪼개기
/pdca	한 기능을 계획·설계·구현·검증·보고 흐름으로 진행	만들고 끝내지 않고 설계와 결과를 대조하기
/control	워크플로가 어느 지점에서 사용자 확인을 받을지 조절	자율성 다이얼

```
# 설치 예시 — 실제 이름과 요구 버전은 저장소에서 다시 확인
claude plugin install bkit
```

Tip

bkit의 병렬 팀 기능을 사용할 때는 Agent Team 실험 플러그가 필요할 수 있습니다. Claude Code와 bkit의 요구 버전을 모두 확인한 뒤 설치하세요.

7.2 Playwright MCP — 브라우저 구조를 직접 제어

Playwright MCP는 Microsoft의 Playwright를 MCP 서버로 연결해 브라우저를 탐색하고 조작할 수 있게 합니다. 화면 좌표만 보고 클릭하는 방식보다 웹 페이지의 접근성 구조와 요소 정보를 활용하므로, 웹 테스트와 반복 브라우저 작업에서 안정적인 경우가 많습니다. Playwright MCP를 클로드에 설치하려면 다음 명령어를 실행합니다.

```
claude mcp add playwright npx @playwright/mcp@latest
```

구분	화면 기반 Computer Use	Playwright MCP
방식	스크린샷을 보고 마우스·키보드처럼 조작	브라우저 요소와 구조를 읽고 명령
범위	브라우저를 포함한 다양한 앱	웹 브라우저 자동화에 특화
장점	사람이 할 수 있는 다양한 UI 작업	웹 테스트의 재현성과 요소 단위 제어
주의	화면 변화와 위치에 영향을 받음	Node.js와 브라우저, MCP 설정이 필요하고 사이트 정책을 지켜야 함

Playwright MCP를 연결했다고 해서 모든 사이트를 자유롭게 자동화해도 된다는 뜻은 아닙니다. 로그인 정보, 개인 정보, 결제, 서비스 약관, 자동화 금지 정책을 확인하고 사용해야 합니다.

7.3 별도 대시보드 없이 /usage와 상태 표시 활용

예전에는 claude-dashboard를 설치해 사용량을 확인할 수도 있었지만, 현재는 유지 상태를 확인하기 어려워 이 책에서는 제외했습니다. 기본으로 제공하는 /usage만으로도 세션 비용, 플랜 사용량, 활동 통계를 확인할 수 있으며 /cost와 /stats도 같은 기능의 별칭입니다.

플랜에 따라 스킬, Sub Agent, 플러그인, MCP 서버별 사용량 분석도 볼 수 있습니다. 터미널 하단에 더 많은 정보를 계속 표시하고 싶다면 Claude Code의 status line 설정이나 검증된 플러그인을 별도로 사용할 수 있지만, 이 책에서는 유지 여부가 불확실한 기능은 소개하지 않습니다.

사용량이 궁금할 때는 클로드 코드 세션에서 다음 명령을 입력하세요.

```
/usage
```

Tip

사용량 화면은 비용을 줄여 주는 기능이 아니라 현재 소비량을 보여 주는 기능입니다. 큰 Agent Team이나 높은 effort 설정을 사용할 때 전후의 사용량을 비교해 보면 어떤 작업이 비용을 많이 쓰는지 쉽게 파악할 수 있습니다.

8 마치며

이번 부록에서는 Claude Code 터미널의 기본 명령어와 권한 모드부터 /effort, ultrathink, Sub Agent, Agent Team, Hooks, Auto-memory, 백그라운드 작업, 여러 세션, 플러그인까지 살펴보았습니다. 마지막으로 각 기능이 해결하는 문제를 중심으로 다시 정리해 보겠습니다.

- claude는 대화형 세션을 열고, claude -p는 한 번 답한 뒤 종료합니다.
- 권한 모드는 승인 횟수를 조절하고, /effort는 사고 강도를 조절합니다.
- ultrathink는 한번의 복잡한 요청에 더 깊은 검토를 요구하는 특별 키워드입니다.
- Sub Agent는 집중 작업을 위임하는 보조 작업자이고, Agent Team은 직접 소통하는 독립 세션들의 팀입니다.
- Agent Team은 실험 플래그를 켜야 하며, 팀원 3~5명과 명확한 파일 경계에서 시작하는 것이 좋습니다.
- Hooks는 Claude의 작업 전후뿐 아니라 세션, 도구, 팀 이벤트에 자동 규칙을 연결합니다.
- CLAUDE.md는 사용자가 쓰는 규정집이고, Auto-memory는 Claude가 축적하는 작업 메모입니다.
- 플러그인은 편리하지만 실제 코드와 권한을 포함할 수 있으므로 출처와 실행 내용을 확인해야 합니다.

코워크와 Claude Code는 서로 경쟁하는 도구가 아닙니다. 코워크가 복잡한 설정을 감춘 자동 변속기라면, Claude Code는 세부 동작과 권한을 직접 조정할 수 있는 수동 변속기에 가깝습니다. 터미널을 매일 사용하지 않더라도 뒤에 서 어떤 구조로 일이 진행되는지 이해하면, 코워크를 사용할 때도 더 정교하게 작업을 설계할 수 있습니다.

이 부록에 있는 모든 기능을 한 번에 익힐 필요는 없습니다. 새로운 프로젝트에서는 /init과 Plan부터, 반복 작업에는 CLAUDE.md와 Hooks부터, 복잡한 병렬 검토가 필요할 때는 Sub Agent와 Agent Team부터 꺼내 쓰면 됩니다.

참고 자료

이번 부록은 아래 자료를 기준으로 집필했습니다. Claude Code는 빠르게 업데이트되므로, 최신 내용은 공식 문서를 함께 확인하기 바랍니다.

1. Anthropic, Claude Code changelog: <https://code.claude.com/docs/en/changelog>
2. Anthropic, CLI reference: <https://code.claude.com/docs/en/cli-reference>
3. Anthropic, Commands: <https://code.claude.com/docs/en/commands>
4. Anthropic, Interactive mode: <https://code.claude.com/docs/en/interactive-mode>
5. Anthropic, Permission modes: <https://code.claude.com/docs/en/permission-modes>
6. Anthropic, Model configuration: <https://code.claude.com/docs/en/model-config>
7. Anthropic, Create custom subagents: <https://code.claude.com/docs/en/sub-agents>
8. Anthropic, Agent teams: <https://code.claude.com/docs/en/agent-teams>
9. Anthropic, Hooks guide: <https://code.claude.com/docs/en/hooks-guide>
10. Anthropic, Memory: <https://code.claude.com/docs/en/memory>
11. Anthropic, Plugins reference: <https://code.claude.com/docs/en/plugins-reference>
12. popup-studio-ai, bkit-claude-code (제3자): <https://github.com/popup-studio-ai/bkit-claude-code>
13. Microsoft, Playwright MCP: <https://github.com/microsoft/playwright-mcp>